# MACHINE LEARNING

Beginner-to-Advanced Learning Guide

Simple Roman Urdu + English Technical Terms + Python Examples

Learn the concepts. Build the models. Complete real projects.

A practical study resource for learners who know basic Python or are learning it alongside ML.

How to use: Har chapter order mein parho, code khud run karo, aur practice tasks complete karo. Sirf read karna kaafi nahi — ML practice se samajh aati hai.

Prepared: November 2026

# Contents


- 1. Machine Learning kya hai?
- 2. AI, ML aur Deep Learning ka farq
- 3. Machine Learning ki 3 main types
- 4. Python aur required libraries
- 5. ML workflow
- 6. Data preprocessing
- 7. Regression algorithms
- 8. Classification algorithms
- 9. Clustering aur PCA
- 10. Model evaluation
- 11. Overfitting aur underfitting
- 12. House Price Prediction project
- 13. Portfolio project roadmap
- 14. 30-day study plan
- 15. Glossary aur practice questions

# 1. Machine Learning kya hai?

Machine Learning (ML) AI ki aik branch hai jahan computer ko har rule manually program karne ke bajaye data se patterns seekhne diye jate hain. Model examples dekhta hai, patterns learn karta hai, aur phir naye data par prediction ya decision deta hai.

## Simple example

Socho tumhare paas 1,000 houses ka data hai: area, bedrooms, location aur price. ML model purane examples se relationship seekhega. Jab tum usay aik naye ghar ka area aur bedrooms doge, woh estimated price predict kar sakta hai.

Yaad rakho: ML magic nahi hai. Model utna hi useful hoga jitna relevant, clean aur representative data usay milega.

## ML kahan use hoti hai?

- Recommendations: YouTube ya shopping apps par relevant items suggest karna.
- Spam detection: Email ko spam ya normal classify karna.
- Fraud detection: Suspicious transactions identify karna.
- Forecasting: Sales, demand ya house prices estimate karna.
- Computer vision: Images mein objects detect ya classify karna.

# 2. AI, ML aur Deep Learning ka farq

Artificial Intelligence (AI) broad field hai. Machine Learning AI ka subset hai, aur Deep Learning ML ka subset hai jo multi-layer neural networks use karta hai.

| Term | Simple meaning | Example |
|---|---|---|
| AI | Machines ko intelligent tasks karwana | Rule-based assistant |
| ML | Data se patterns learn karna | Spam classifier |
| Deep Learning | Neural networks ki multiple layers se learning | Image recognition |

Har ML problem ko Deep Learning ki zaroorat nahi hoti. Small structured datasets ke liye traditional ML aksar simpler aur effective hoti hai.

# 3. Machine Learning ki 3 main types

## A. Supervised Learning

Is mein training data ke saath correct answer/label bhi hota hai. Model input aur answer ka relation seekhta hai.

- Regression: number predict karna, jaise house price ya temperature.
- Classification: category predict karna, jaise spam/not spam.

## B. Unsupervised Learning

Data ke labels nahi hote. Model khud groups ya hidden structure discover karta hai.

- Clustering: customers ko behavior ke hisaab se groups mein divide karna.
- Dimensionality reduction: bohat zyada features ko fewer useful dimensions mein represent karna.

## C. Reinforcement Learning

Agent environment mein actions leta hai aur rewards ya penalties se seekhta hai. Example: game-playing agent ya robot navigation.

Quick rule: Answer available ho = supervised. Labels na hon = unsupervised. Actions + rewards hon = reinforcement learning.

# 4. Python aur required libraries

ML ke liye Python popular hai kyun ke iska ecosystem strong aur beginner-friendly hai. Python basics, functions, lists/dictionaries aur basic OOP samajhna helpful hai.

| Library | Use |
| --- | --- |
| NumPy | Numerical arrays aur mathematical operations |
| Pandas | CSV/data tables read, clean aur analyze karna |
| Matplotlib | Graphs aur data visualization |
| Seaborn | Statistical visualizations |
| Scikit-learn | Traditional ML models, preprocessing aur evaluation |
| Jupyter Notebook | Code ko cells mein run aur document karna |

## Installation

## PYTHON EXAMPLE

```
python -m venv .venv

# Windows:
.venv\\Scripts\\activate

# macOS / Linux:
source .venv/bin/activate

pip install numpy pandas matplotlib seaborn scikit-learn jupyter
jupyter notebook
```

## Pandas mini example

## PYTHON EXAMPLE

```
import pandas as pd

data = {
    "hours_studied": [1, 2, 3, 4],
    "marks": [45, 52, 67, 78]
}

df = pd.DataFrame(data)
print(df.head())
print(df.describe())
```

# 5. ML workflow: problem se model tak

- 1. Problem define karo: exactly kya predict ya discover karna hai?
- 2. Data collect karo: reliable source se relevant examples lo.
- 3. Data inspect karo: columns, data types, missing values aur duplicates check karo.
- 4. Preprocess karo: missing values handle, categories encode, scaling where needed.
- 5. Split karo: training aur testing data alag rakho.
- 6. Baseline banao: simple model se starting performance measure karo.
- 7. Train karo: model ko training data par fit karo.
- 8. Evaluate karo: suitable metrics se unseen test data par check karo.

- 9. Improve karo: errors analyze karo, features ya parameters improve karo.
- 10. Deploy aur monitor karo: real use mein performance aur data changes monitor karo.

Important: Test data ko model selection ke dauran baar baar use mat karo. Tuning ke liye validation set ya cross-validation use karo; final test set ko end par rakho.

# 6. Data preprocessing

## Missing values

Missing values ko blindly zero se replace nahi karna chahiye. Numeric column ke liye median aur categorical column ke liye most frequent value useful starting choices ho sakti hain. Choice data aur problem par depend karti hai.

## Categorical encoding

Models ko aksar text categories numeric form mein chahiye hoti hain. One-hot encoding categories ko separate binary columns mein convert karti hai.

## Feature scaling

Standardization features ko comparable scale par laati hai. KNN, SVM aur many gradient-based models ko scaling se faida hota hai. Decision trees aur random forests aam tor par scaling ke baghair bhi kaam kar lete hain.

## Train-test split

### PYTHON EXAMPLE

```
from sklearn.model_selection import train_test_split

X = df[["hours_studied"]]
y = df["marks"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Data leakage se bacho: Imputation, scaling aur feature selection jaisi steps sirf training data par fit honi chahiye. Scikit-learn Pipeline is process ko safe banane mein madad karti hai.

# 7. Regression algorithms

Regression tab use hoti hai jab target aik numeric value ho, jaise price, sales ya delivery time.

| Algorithm | Basic idea |
|---|---|
| Linear Regression | Input aur output ke darmiyan linear relationship fit karta hai. |
| Decision Tree Regressor | Rules/splits ke zariye numeric prediction karta hai. |
| Random Forest Regressor | Multiple decision trees ke predictions combine karta hai. |
| Gradient Boosting | Models ko sequence mein train karke previous errors improve karta hai. |

## Linear Regression example

### PYTHON EXAMPLE

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

```
print(predictions)
```

Model.fit() training karta hai. Model.predict() naye inputs ke liye output estimate karta hai. Real project mein data split aur preprocessing bhi zaroori hai.

# 8. Classification algorithms

Classification tab use hoti hai jab output category/label ho, jaise fraud/not fraud, spam/not spam ya customer churn/retained.

| Algorithm | Basic idea |
|---|---|
| Logistic Regression | Class probabilities estimate karta hai. |
| K-Nearest Neighbors | Nearby training examples ke labels dekh kar class decide karta hai. |
| Decision Tree | Feature-based rules se class predict karta hai. |
| Random Forest | Multiple trees ki predictions combine karta hai. |
| Support Vector Machine | Classes ke darmiyan separating boundary find karta hai. |

## Classification example

## PYTHON EXAMPLE

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2,
    random_state=42, stratify=iris.target
)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
```

# 9. Clustering aur dimensionality reduction

## K-Means clustering

K-Means unlabeled data ko K groups mein divide karta hai. Algorithm cluster centers assign/update karta hai. K choose karna important hai; elbow method aik exploratory technique hai, final proof nahi.

## PYTHON EXAMPLE

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# X = numeric feature columns
X_scaled = StandardScaler().fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42, n_init="auto")
clusters = kmeans.fit_predict(X_scaled)
```

## PCA

Principal Component Analysis (PCA) features ko fewer dimensions mein project karta hai, while trying to preserve as much variance as possible. PCA se pehle scaling aksar important hoti hai.

# 10. Model evaluation

Sirf aik metric dekh kar model ko best declare mat karo. Metric ka selection business problem aur error ki cost ke mutabiq hota hai.

| Metric | Meaning / when useful |
|---|---|
| Accuracy | Correct predictions ka proportion. Imbalanced data par misleading ho sakti hai. |
| Precision | Predicted positives mein se kitne actually positive thay. |
| Recall | Actual positives mein se kitne detect hue. |
| F1-score | Precision aur recall ka harmonic mean. |
| Confusion Matrix | True/false positives aur negatives ka breakdown. |
| MAE | Regression mein average absolute error. |
| MSE / RMSE | Squared error; RMSE original target units mein hota hai. |

## Metrics example

## PYTHON EXAMPLE

```
from sklearn.metrics import classification_report, confusion_matrix

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
```

# 11. Overfitting, underfitting aur improvement

## Underfitting

Model bohat simple hai aur training data ke patterns bhi properly learn nahi kar pa raha. Training aur test dono performance weak hoti hai.

## Overfitting

Model training data ko bohat zyada memorize kar leta hai. Training performance high hoti hai lekin unseen test data par weak ho sakta hai.

## Improve kaise karein?

- Zyada relevant aur representative data collect karo.
- Data leakage aur incorrect labels check karo.
- Model complexity control karo; regularization ya pruning use karo.
- Cross-validation se performance estimate karo.
- Hyperparameters ko validation data par tune karo.
- Error analysis karo: model kis type ki examples par fail hota hai?

# 12. Complete beginner project: House Price Prediction

Goal: house ke features se price estimate karna. Is project mein tum regression workflow practice karoge. Example scikit-learn ke California housing dataset ko use karta hai.

## Step 1 — imports aur data

### PYTHON EXAMPLE

```
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
X = housing.data
y = housing.target

print(X.head())
print(y.head())
```

### Step 2 — split, train aur predict

### PYTHON EXAMPLE

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
model.fit(X_train, y_train)

predictions = model.predict(X_test)
mae = mean_absolute_error(y_test, predictions)
rmse = mean_squared_error(y_test, predictions) ** 0.5
print("MAE:", mae)
print("RMSE:", rmse)
```

Note: California housing dataset ka target documented units mein hota hai; output ko dollars samajh kar label mat karna. Is exercise ka focus workflow hai, production-grade real-estate valuation nahi.

### Step 3 — practice

- Ridge ke bajaye LinearRegression aur RandomForestRegressor try karo.
- MAE aur RMSE compare karo.
- Residuals (actual minus predicted) ka plot banao.
- Explain karo ke model ki prediction mein kaun se limitations hain.

# 13. Portfolio projects roadmap

| Level | Project | What you learn |
|---|---|---|
| Beginner | Iris flower classifier | Classification, split, accuracy |
| Beginner | Student marks prediction | Regression, MAE, visualization |
| Intermediate | Customer segmentation | Scaling, K-Means, PCA |
| Intermediate | Email spam detector | Text preprocessing, classification |
| Advanced | Customer churn prediction | Imbalanced data, recall, pipelines |
| Advanced | Sales forecasting | Time-aware validation, forecasting features |

Portfolio tip: Har project ke GitHub README mein problem, dataset source, preprocessing, model choice, metrics, limitations aur run instructions likho. Sirf accuracy number publish na karo.

# 14. 30-day study plan

| Days | Focus | Daily output |
|---|---|---|
| 1–3 | ML basics + types | Concepts apne words mein explain karo |
| 4–6 | NumPy + Pandas | CSV load, inspect, clean |
| 7–9 | Matplotlib + analysis | 3 useful charts banao |
| 10–12 | Preprocessing + split | Clean dataset pipeline |
| 13–16 | Regression | House/student prediction model |
| 17–20 | Classification | Iris/spam classifier |
| 21–23 | Clustering + PCA | Customer groups explore karo |
| 24–26 | Evaluation + overfitting | Metrics compare, errors analyze |
| 27–30 | Final mini project | GitHub repo + README + demo |

Suggested daily routine: 20 minutes concept, 40 minutes coding, 20 minutes practice/review. Agar kisi din time kam ho, kam az kam code khud run karo aur aik short note likho.

# 15. Glossary

Dataset: Examples/records ka collection.

Feature: Input column ya variable jo prediction mein use hota hai.

Label / Target: Woh answer jo supervised model predict karta hai.

Model: Learned mathematical pattern/system.

Training: Data se model parameters learn karne ka process.

Inference: Trained model se new input par prediction lena.

Epoch: Neural network training mein complete training dataset ka aik pass.

Hyperparameter: Training se pehle set ki jane wali configuration.

Cross-validation: Data ke multiple splits par model performance check karna.

Data leakage: Jab training process ko aisi information mil jaye jo prediction time par available nahi honi chahiye.

## Practice questions

- Machine Learning aur rule-based programming mein kya farq hai?
- Regression aur classification ka aik aik real-world example do.
- Train aur test data alag kyun rakhte hain?
- Accuracy imbalanced dataset par misleading kyun ho sakti hai?
- Overfitting ki do signs aur do possible solutions batao.
- Data leakage kya hai? Aik example do.
- K-Means kis type ke problem ke liye use hota hai?
- House price project mein MAE ka matlab apne words mein samjhao.

Final challenge: Aik dataset choose karo, problem define karo, data explore aur clean karo, baseline model train karo, suitable metrics se evaluate karo, aur GitHub par complete README ke saath publish karo.

# Learning resources

- Scikit-learn User Guide: https://scikit-learn.org/stable/user_guide.html

- Pandas Getting Started: https://pandas.pydata.org/docs/getting_started/
- NumPy Learn: https://numpy.org/learn/
- Matplotlib Tutorials: https://matplotlib.org/stable/tutorials/
- Google Machine Learning Crash Course: https://developers.google.com/machine-learning/crash-course

Use official resources for updated APIs and deeper explanations. Library versions change, so current documentation check karo agar code examples different behave karein.

End of guide — keep building, not just reading.